SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

CSA37 SOFTWARE TESTING — ASSIGNMENT 3

Student Name: Yeturu Thanvi | **Register Number:** 192371026

Case Study: AgriMart Farmer-to-Buyer Marketplace | **Course Outcome:** CO3 (Unit, Integration, System, Regression, Acceptance Testing)

TASK 1 – Evaluate the Scope of Unit Testing (5 Marks)

1. Unit Definition for AgriMart:
Treating each screen of the app as a 'unit' is fundamentally flawed because screens combine UI rendering, state management, validation logic, and API calls into a single coupled component. For AgriMart, a **unit** must be defined as the smallest testable piece of source code—specifically, individual functions, classes, or methods.
• *Examples in M3 (Price Engine):* The function calculateSuggestedPrice(marketPrice, qualityGrade) or the timeout handler fetchMarketPriceWithTimeout().
• *Examples in M4 (Cart and Order):* The function calculateOrderTotal(items, discount, deliveryFee) or state validation rules.

2. Goals of Unit Testing: To isolate individual modules, verify functional correctness against specifications, ensure logic paths execute correctly, and detect defects early before integration.

3. Defects Found & Not Found:
• *Can find:* Algorithmic errors, boundary condition failures (e.g., ±15% price limits), incorrect quality factor multipliers (e.g., Grade C 0.85), and exception handling.
• *Cannot find:* Interface mismatches between M3 and M4, network latency issues, database locking conflicts, and end-to-end payment gateway failures.

4-5. Comparison & Recommendation: Comparing 'screen as unit' vs 'function/class as unit' reveals that screens obscure underlying logic errors behind UI frameworks. *Recommendation:* Define units at the **function/class level** with mocked dependencies, ensuring complete test coverage of core business logic.

TASK 2 – Evaluate Unit Test Planning (2 Marks)

1. Unit Test Plan Contents for M3 (Price Engine):
• **Objectives:** Verify that suggested prices conform to market rates multiplied by quality factors (Grade A: 1.10, Grade B: 1.00, Grade C: 0.85), enforce the ±15% farmer pricing bounds, and handle API timeouts (>3 seconds) by utilizing fallback prices (<24 hours old).
• **Features to Be Tested:** Price calculation module, API timeout fallback logic, and validation rules.
• **Approach & Pass/Fail Criteria.** Automated unit testing via pytest/JUnit with mock objects. Pass criteria: 100% pass rate on test vectors covering normal bounds, boundary limits, and timeout exceptions.
• **Resources & Time:** 1 Test Engineer, 2 days of execution, Python pytest framework.
• **Risks of Skipping:** Skipping unit test planning for M3 would result in unchecked pricing errors, financial loss for farmers, and transaction disputes.

TASK 3 – Design and Evaluate Unit Tests for the Price Engine (6 Marks)

1-5. Unit Test Suite for Price Engine (M3):
Testing suggested price calculations, ±15% bounds, API response times, and fallback rules (updated for change C1: Grade C factor = 0.85).

Test ID	Test Category	Input Parameters (Market Price, Grade, Farmer Price)	Expected Result	Status
TC_M3_01	Positive (Grade A)	Market: ■2000, Grade A, Farmer: ■2200	Suggested: ■2200. Valid (within ±15% of ■2200)	Pass
TC_M3_02	Positive (Grade C - C1)	Market: ■1000, Grade C, Farmer: ■850	Suggested: ■850 (1000 × 0.85). Valid.	Pass

192371026 YETURU THANVI

TC_M3_03	Boundary (Upper +15%)	Market: ■1000, Grade B (1000), Farmer: ■1150	Valid (Exactly +15% upper bound)	Pass
TC_M3_04	Boundary (Invalid Upper)	Market: ■1000, Grade B (1000), Farmer: ■1151	Rejected (> +15% limit)	Pass
TC_M3_05	Exception (API Timeout)	API response delay = 3.5s, Fallback price age = 12h	Uses fallback price successfully	Pass
TC_M3_06	Exception (Stale Fallback)	API delay = 3.5s, Fallback price age = 26h	Listing blocked (> 24h old)	Pass

Sufficiency Judgment: This test suite achieves 100% branch coverage and boundary condition verification, ensuring no farmer is shown an unfair price.

TASK 4 – Design and Evaluate the Test Harness (4 Marks)

1-4. Test Harness for Price Engine (M3):
• **Drivers & Stubs:** Since M3 depends on external market-price API and M2 (Produce Listing), we need a **Test Driver** to invoke M3 functions and a **Mock Stub** to simulate the market-price API.
• **Stub Simulation:** The stub must simulate: (a) normal response within <3s, (b) slow response (>3s timeout), (c) HTTP failure / downtime, and (d) varying fallback timestamps (<24h vs >24h).
• **Test Data & Storage:** Stored in JSON/CSV files containing test vectors and expected outputs.
• **Risks & Mitigation:** Stubs may not accurately reflect real-world network latency. *Mitigation:* Periodically validate stubs against staging API responses.

TASK 5 – Evaluate Integration Strategies (3 Marks)

Module Status Analysis: M1 (Complete), M2 (Complete), M3 (Complete), M4 (In Progress), M5 (Not Started), M6 (Not Started), M7 (In Progress).
• **Big-Bang:** Inappropriate; links unbuilt modules (M5, M6), leading to untraceable integration failures.
• **Top-Down:** Requires extensive stubs for incomplete lower modules.
• **Bottom-Up:** Highly suitable. Completed modules (M1, M2, M3) can be integrated and tested first as a solid foundation before adding M4 and M7.
• *Recommendation:* **Bottom-Up Integration Strategy** combined with sandwich testing for active modules.

TASK 6 – Construct and Evaluate the Integration Test Plan (3 Marks)

Integration Order & Criteria:
• *Step 1:* Integrate M1 (Registration/KYC) + M2 (Produce Listing). Prove farmer onboarding and produce listing linkage.
• *Step 2:* Integrate M3 (Price Engine) with M2. Test suggested price generation using market API stubs.
• *Step 3:* Integrate M4 (Cart & Order) as it progresses, linking to M2 and M3.
• *Entry/Exit Criteria:* Entry: Unit tests passed (100%). Exit: Interface data flows verified with zero blocking defects.

TASK 7 – Evaluate System Testing Needs (4 Marks)

Essential vs. Reduced System Testing:
• **Usability Testing (Essential):** Vital for farmers with limited literacy and regional language requirements (Tamil voice prompts - C3).
• **Performance/Load Testing (Essential):** Must handle 25,000 farmers and 3,000 buyers on slow rural networks.
• **Compatibility Testing (Essential):** Must operate smoothly on low-cost Android phones.
• **Security Testing (Essential):** Protects Aadhaar e-KYC (M1) and UPI payments (M5).
• *Reducible:* Advanced multi-browser desktop compatibility (AgriMart is primarily mobile-focused).

192371026 YETURU THANVI

TASK 8 – Evaluate Regression Testing (4 Marks)

Regression Traceability for Changes C1–C4:
• **C1 (Grade C factor 0.80 → 0.85 in M3):** Retest Price Engine calculations for Grade C produce.
• **C2 (Order cancellation window 2h → 1h in M4):** Retest order state machine and timer triggers.
• **C3 (New Tamil voice prompts added in M7):** Retest audio notification triggers and localization.
• **C4 (Fix for duplicate farmer registration in M1):** Retest mobile number uniqueness validation.
• *Strategy:* Risk-based selective regression testing combined with automated test suites.

TASK 9 – Evaluate Alpha and Beta Testing (3 Marks)

Alpha & Beta Testing Plan:
• **Alpha Testing:** Conducted internally by agricultural department staff and QA engineers in simulated rural network conditions.
• **Beta Testing:** Deployed to 100 real farmers across 5 rural districts for 2 weeks.
• **Feedback Collection:** Voice-recorded feedback sessions and simplified pictorial reporting for farmers with limited literacy to evaluate Tamil voice prompts (C3) and low-end phone usability.

TASK 10 – Evaluate Acceptance Testing (4 Marks)

Acceptance Criteria & Sign-off:
• **Measurable Criteria:** (1) Price calculation completes within 3 seconds or rolls back correctly; (2) App loads within 4 seconds on low-cost 2G/3G phones; (3) Zero unhandled exceptions during 500 concurrent transactions.
• **Scenarios:** Farmer registers via Aadhaar, lists produce, checks price, buyer places order, payment completes via UPI.
• **Sign-off:** Jointly signed off by the State Agriculture Department Director and Lead Test Manager.

TASK 11 – Recommend and Defend the Overall Testing Strategy (4 Marks)

Overall Testing Strategy Recommendation:
A multi-level testing approach (Unit testing for business logic, Bottom-Up integration for modules M1–M4, rigorous System testing for low-end devices and slow networks, Risk-based regression for C1–C4, and Beta testing with real farmers) ensures high quality, reliability, and successful adoption across rural farming communities.

TASK 12 – Day-4 Design Challenge: Re-evaluate and Adapt the Testing Strategy (8 Marks)

A. Impact Analysis: Integration with national e-NAM trading platform, hotfix release for M5 within 1 week, and e-NAM test environment available only 2 hours a day.
B. Evaluation of Original Strategy: Original bottom-up strategy requires modification because e-NAM API access is severely restricted (2 hours/day) and the M5 hotfix demands immediate risk-based regression.
C. Adaptation & Redesign:
• *e-NAM Simulator:* Build robust virtual e-NAM stubs for 22 hours/day; restrict live e-NAM integration testing to the allocated 2-hour window.
• *Hotfix Regression:* Execute a targeted smoke and regression test set focused on M5 (UPI payment gateway) and M4 (Cart/Order) within 24 hours.
D. Evidence-Based Defence: This adapted strategy respects strict schedule constraints, mitigates environment unavailability risks, and guarantees payment reliability.

192371026 YETURU THANVI